



Βάσεις Δεδομένων

Εργασία Εξαμήνου – 2^ο Παραδοτέο

(ΑΠΑΝΤΗΣΕΙΣ CHATGPT ΚΑΙ ΣΧΟΛΙΑΣΜΟΣ)

Αριθμός Ομάδας: 4

- **Οδυσσέας Μουρελάτος (8240080)**
- **Αναστάσης Χατζηδάκης (8240165)**

Καθηγητές:

- **Δαμιανός Χατζηαντωνίου**
- **Σταύρος Γρηγορακάκης**

ΠΕΡΙΕΧΟΜΕΝΑ

1. ΑΠΑΝΤΗΣΕΙΣ CHATGPT.....	3
2. ΔΙΚΕΣ ΜΑΣ ΑΠΑΝΤΗΣΕΙΣ.....	5
3. ΣΧΟΛΙΑΣΜΟΣ ΑΠΑΝΤΗΣΕΩΝ CHATGPT & ΔΙΑΦΟΡΕΣ.....	7
3.1 ΕΡΩΤΗΜΑ 9.....	7
3.2 ΕΡΩΤΗΜΑ 13.....	7
3.3 ΕΡΩΤΗΜΑ 14.....	8

1. ΑΠΑΝΤΗΣΕΙΣ CHATGPT

Με βάση το σχεσιακό μοντέλο, λύσε MONO τα ερωτήματα με @

Με βάση αποκλειστικά το δοσμένο σχεσιακό μοντέλο, τα ερωτήματα με @ είναι τα 9, 13 και 14. Παρακάτω δίνω καθαρές SQL λύσεις, με λογική που ταιριάζει σε εξεταστικό πλαίσιο.

9) Για κάθε πελάτη και κάθε μήνα του 2017, όνομα πελάτη και άθροισμα ποσού συναλλαγών

👉 Χρησιμοποιούμε `Customer` + `CreditCard` + `CardTransaction`

sql

📋 Αντιγραφή κώδικα

```
SELECT
    c.firstName,
    c.lastName,
    EXTRACT(MONTH FROM t.date) AS month,
    SUM(t.chargeAmount) AS totalAmount
FROM Customer c
JOIN CreditCard cc ON c.custCode = cc.custCode
JOIN CardTransaction t ON cc.cardNumber = t.cardNumber
WHERE EXTRACT(YEAR FROM t.date) = 2017
GROUP BY
    c.firstName,
    c.lastName,
    EXTRACT(MONTH FROM t.date)
ORDER BY c.lastName, month;
```



13) Πελάτες που τον Ιούνιο 2017 είχαν μέσο ποσό αγορών $\geq 3 \times$ τον γενικό μέσο όρο του μήνα

Βήμα 1: Γενικός μέσος όρος Ιουνίου 2017

Βήμα 2: Μέσος όρος ανά πελάτη

Βήμα 3: Σύγκριση

sql

 Αντιγραφή κώδικα

```
SELECT
    c.firstName,
    c.lastName
FROM Customer c
JOIN CreditCard cc ON c.custCode = cc.custCode
JOIN CardTransaction t ON cc.cardNumber = t.cardNumber
WHERE EXTRACT(YEAR FROM t.date) = 2017
    AND EXTRACT(MONTH FROM t.date) = 6
GROUP BY c.custCode, c.firstName, c.lastName
HAVING AVG(t.chargeAmount) >= 3 * (
    SELECT AVG(t2.chargeAmount)
    FROM CardTransaction t2
    WHERE EXTRACT(YEAR FROM t2.date) = 2017
    AND EXTRACT(MONTH FROM t2.date) = 6
);
```



14) Κωδικοί πελατών που αύξησαν τις αγορές Ιούνιο 2017 $\geq 50\%$ σε σχέση με Ιούνιο 2016

👉 Σύγκριση SUM(2017) με SUM(2016) ανά πελάτη

sql

 Αντιγραφή κώδικα

```
SELECT
    c.custCode
FROM Customer c
JOIN CreditCard cc ON c.custCode = cc.custCode
JOIN CardTransaction t ON cc.cardNumber = t.cardNumber
WHERE EXTRACT(MONTH FROM t.date) = 6
GROUP BY c.custCode
HAVING
    SUM(CASE WHEN EXTRACT(YEAR FROM t.date) = 2017 THEN t.chargeAmount ELSE 0 END)
    >=
    1.5 * SUM(CASE WHEN EXTRACT(YEAR FROM t.date) = 2016 THEN t.chargeAmount ELSE 0 END);
```



2.ΔΙΚΕΣ ΜΑΣ ΑΠΑΝΤΗΣΕΙΣ

```
--9) Εμφάνισε για κάθε πελάτη και κάθε μήνα του 2017, το όνομα του
-- συναλλαγών του και το άθροισμα του ποσού των συναλλαγών του.
SELECT firstName, lastName, month(date) as month, SUM(chargeAmount) as sum
FROM Customer C, CreditCard Cr, CardTransaction Ct
WHERE C.custCode = Cr.custCode AND
      Cr.cardNumber = Ct.cardNumber AND
      year(date) = 2017
GROUP BY firstName, lastName, month(date)|

--13) Δείξε το όνομα των πελατών που τον Ιούνιο του 2017 είχαν μέσο ποσό αγορών τρεις φορές
-- μεγαλύτερο από το μέσο ποσό αγορών αυτού του μήνα (για όλους τους πελάτες).
CREATE VIEW V13_1(custCode, average) AS
SELECT Cr.custCode, AVG(chargeAmount)
FROM CreditCard Cr, CardTransaction Ct
WHERE Cr.cardNumber = Ct.cardNumber AND
      MONTH(date) = 6 AND
      year(date) = 2017
GROUP BY cr.custCode

SELECT firstName, lastName
FROM Customer C
WHERE C.custCode IN (SELECT V13_1.custCode
                     FROM V13_1
                     WHERE (V13_1.average/3) > (SELECT AVG(chargeAmount)
                                                  FROM CardTransaction Ct2
                                                  WHERE month(Ct2.date)=6 AND
                                                        year(Ct2.date)=2017))

--14) Εμφάνισε τον κωδικό των πελατών που αύξησαν τις συνολικές τους αγορές τον Ιούνιο του 2017
-- τουλάχιστον κατά 50% σε σχέση με τον Ιούνιο του 2016.
CREATE VIEW V14_1(custCode, sum17) AS
SELECT Cr.custCode, SUM(chargeAmount)
FROM CreditCard Cr, CardTransaction Ct
WHERE Cr.cardNumber = Ct.cardNumber AND
      MONTH(date) = 6 AND
      year(date) = 2017
GROUP BY cr.custCode

CREATE VIEW V14_2(custCode, sum16) AS
SELECT Cr.custCode, SUM(chargeAmount)
FROM CreditCard Cr, CardTransaction Ct
WHERE Cr.cardNumber = Ct.cardNumber AND
      MONTH(date) = 6 AND
      year(date) = 2016
GROUP BY cr.custCode

SELECT V14_1.custCode
FROM V14_1, V14_2
WHERE V14_1.custCode = V14_2.custCode
      AND (sum17/sum16)>=1.5
```

```
-- Χωρίς την χρήση views το ερώτημα θα λυνόταν ως εξής
SELECT T1.custCode
FROM (SELECT Cr.custCode AS custCode, SUM(chargeAmount) as sum17
      FROM CreditCard Cr, CardTransaction Ct
      WHERE Cr.cardNumber = Ct.cardNumber AND
            MONTH(date) = 6 AND
            year(date) = 2017
      GROUP BY cr.custCode) AS T1,
      (SELECT Cr.custCode AS custCode, SUM(chargeAmount) as sum16
      FROM CreditCard Cr, CardTransaction Ct
      WHERE Cr.cardNumber = Ct.cardNumber AND
            MONTH(date) = 6 AND
            year(date) = 2016
      GROUP BY cr.custCode) AS T2
WHERE T1.custCode = T2.custCode AND
      (sum17/sum16)>=1.5
```

3. ΣΧΟΛΙΑΣΜΟΣ ΑΠΑΝΤΗΣΕΩΝ CHATGPT & ΔΙΑΦΟΡΕΣ

3.1 Ερώτημα 9

Χρήση άγνωστων σε εμάς συναρτήσεων για να πάρουμε τον μήνα και έτος από ένα γνώρισμα τύπου DATETIME. Ίσως εφαρμόζονται σε άλλο server πχ MySQL, όμως στον SQL Server Management Studio που χρησιμοποιούμε, ο σωστός τρόπος είναι είτε όπως το κάναμε εμείς (με MONTH(date) & YEAR(date)) είτε με DATEPART(month, date) & DATEPART(year, date). *

Επιπλέον, ενώνει τους πίνακες με JOIN αντί για συνθήκες ένωσης πινάκων στο WHERE, κάτι το οποίο είναι σωστό και δεν παρουσιάζει κάποιο πρόβλημα. Ωστόσο καθ' όλη την διάρκεια του εξαμήνου στα παραδείγματα συναντάμε συνήθως συνθήκες ένωσης πινάκων στο WHERE, για αυτό και επιλέξαμε αυτή την προσέγγιση.

Επίσης, αναφέρεται ακόμα και σε γνωρίσματα που υπάρχουν μόνο σε ένα πίνακα (πχ firstName, lastName) με το όνομα του πίνακα (C.firstName & C.lastName), κάτι που είναι αποδεκτό ίσως για λόγους σαφήνειας.

Τέλος, χρησιμοποιεί το ORDER BY για να ταξινομήσει τα αποτελέσματα με βάση το επίθετο πρώτα και μετά τον μήνα, που είναι επίσης αποδεκτό.

*Το ίδιο ισχύει και για τα ερωτήματα 13 & 14

3.2 Ερώτημα 13

Προφανώς η πρώτη διαφορά που παρατηρείται είναι ότι δεν χρησιμοποιεί views για ένα ερώτημα όπου η χρήση μιας όψης κρίνεται ιδανική.

Από εκεί και πέρα, το SELECT, η χρήση των πινάκων, η ένωση τους και οι συνθήκες στο WHERE φαίνονται γενικά ότι εφαρμόζονται σωστά (εξαιρώντας το λανθασμένο συντακτικό που αναφέραμε προηγουμένως). Μετά ακολουθεί το group by με βάση τον κωδικό πελάτη και το ονοματεπώνυμο του, και τέλος χάρη στο HAVING εμφανίζει πράγματι μόνο τους πελάτες που οι μέσες αγορές τους τον συγκεκριμένο μήνα είναι 3 φορές μεγαλύτερες από το μέσο ποσό αγοράς του μήνα για όλους τους πελάτες (με

μόνη σημείωση ότι δέχεται και την ισότητα, ενώ στο ερώτημα δεν αναφέρεται πουθενά η λέξη «τουλάχιστον»)

Εμείς αρχικά χρησιμοποιούμε ένα view για να βρούμε το average κάθε πελάτη τον Ιούνιο του 2017. Έπειτα βρίσκουμε εμφωλευμένα το average του μήνα για όλους τους πελάτες (θα μπορούσε και αυτό να γίνει ξεχωριστό view), κάνουμε την σύγκριση για κάθε πελάτη που υπάρχει στο view που δημιουργήσαμε, και επειδή θέλουμε το ονοματεπώνυμο του και όχι τον κωδικό του, εμφανίζουμε τα ονοματεπώνυμα των πελατών με την χρήση του γνωστού τελεστή IN.

Και οι δύο λύσεις θεωρούνται αποδεκτές (με προφανώς αλλαγή συντακτικού συναρτήσεων για γνωρίσματα τύπου DATETIME).

3.3 Ερώτημα 14

Προφανώς εδώ οι διαφορές είναι τεράστιες καθώς χρησιμοποιεί άγνωστο σε εμάς συντακτικό για ένα ερώτημα που λύνεται πολύ πιο απλά με VIEWS ή ακόμα και εμφωλευμένα.

Το SELECT, η χρήση των πινάκων και η ένωση τους φαίνονται γενικά ότι εφαρμόζονται σωστά, ωστόσο από εκεί και πέρα η λύση είναι αδύνατο να περιγραφτεί με ακρίβεια με βάση τις γνώσεις που έχουμε μέχρι στιγμής. Φαίνεται ότι κάνει group με βάση τον κωδικό των εργαζομένων και εμφανίζει μόνο αυτούς που έχουν αυξήσει τις αγορές τους τουλάχιστον κατά 50% τον Ιούνιο του 2017 σε σχέση με τον Ιούνιο του 2016, όπως ακριβώς ζητείτε. Το πρόβλημα προφανώς είναι ότι δεν μας έχει διδαχτεί αυτό το συντακτικό, οπότε δεν είναι δυνατό ούτε να αντιληφθούμε πλήρως την ιδέα της λύσης που προτείνει αλλά ούτε φυσικά και να την εφαρμόσουμε.